Joe Miguel Robertazzi

tazzi@stanford.edu | jtazzi.com | linkedin.com/in/jtazzi | github.com/foxboxxx

Education

Stanford University — Stanford, CA

*B.S. in Computer Science (Systems), Minor in Electrical Engineering — **GPA: 3.9/4.0*** — *Expected June 2027*

M.S. in Computer Science (Coterminal in Artificial Intelligence, applying) — *Expected June 2028*

- **Coursework:** Computer System Architecture, Operating System Design, Computer Systems Programming, Parallel Computing, Digital System Design, Embedded Systems, Computer Networking, HW Accelerators for ML

Experience

Performance Control Software Engineer Intern — June 2026 – Sept. 2026

Apple – Platform Architecture – Performance & Power Control — *Cupertino, CA*

- Developing C++ online scheduling indicators feeding runtime signals into performance and power control on custom silicon
- Benchmarking and profiling post-silicon workloads to surface microarchitectural flags driving scheduling and DVFS tuning
- Partnering with hardware and software teams to translate profiling results into concrete system-level tuning changes

Teaching Assistant (CS107E, CS106A/B) — Apr. 2024 – Present

Stanford University – Department of Computer Science — *Stanford, CA*

- Led weekly bare-metal C labs for 20 students in CS107E; taught Python (CS106A) and C++ (CS106B) sections quarterly
- Authored technical guides and helper libraries on drivers and memory layout, enabling advanced bare-metal final projects
- Ran biweekly grading sessions, debugged student C/C++/Python programs in office hours, and evaluated quarterly exams

Embedded Systems Intern — June 2025 – Aug. 2025

Stanford University – Computer Systems from the Ground Up Course Infrastructure — *Stanford, CA*

- Optimized I2C, SPI, and DMA drivers for 12+ peripherals including OLEDs, cameras, audio codecs, and motion sensors
- Redesigned and stress-tested the I2C and I2S APIs into a unified hardware abstraction layer used across course labs
- Ported legacy course projects to RISC-V, designed custom PCBs, and documented the RISC-V Vector Extension in C/ASM

Independent Computational Researcher | Manuscript — Jan. 2021 – June 2023

Self-directed project, mentored by a faculty advisor at the University of Rochester — *Rochester, NY*

- Built Python (scikit-learn, pygmt) pipelines over 50M+ avian migration points correlating geomagnetic anomalies with population decline; authored manuscript, **Regeneron STS Top 40 Finalist** (of 1,900+)

Projects

Bare-Metal Runtime Debugger | *C, ARM, Raspberry Pi* — Apr. 2026 – June 2026

- Built a two-Pi bare-metal debugger: a GPIO doorbell interrupt hijacks the running target and parks it in a debug handler
- Injected ARM machine code over a bit-banged UART packet protocol, patching arguments into the payloads at runtime
- Enabled single-stepping on hardware with no step mode using CP14 mismatch breakpoints, watchpoints, and fault registers
- Drove memory and GPIO read/write from a full-screen ST7735R LCD UI, navigated by a hand-decoded NEC IR remote

Trading Tamagotchi | *C, Python, ARM, Raspberry Pi* | Demo — Jan. 2026 – Mar. 2026

- Built a handheld bare-metal pet that lives or dies on real Kalshi trades: 6 menus, 40+ hand-drawn frames, and a state machine killing it on a 3-loss streak or 24h without a trade
- Wrote every driver from scratch: SPI for a 128x128 ST7735R LCD, bit-banged I2C for a PCF8523 RTC, interrupt-driven keypad, and a 921600-baud UART bridge to a Python Kalshi API client
- Cut interrupt latency **4,650 → 1,386 cycles (3.4x)** over 5 profiled stages (I-cache, branch prediction, FIQ, inlined GPIO, `-O3`); extended the FAT32 driver with writes to persist state across reboots

GPU Histogram Kernel Optimization | *CUDA, C++, H100 GPU* — Nov. 2025 – Dec. 2025

- Won **1st of 50+** in the CS149 kernel competition, cutting a 512-channel histogram from 6.59ms to 310μs (21x) on an H100
- Cut L1→L2 traffic 16GB → 16.5MB by moving atomics into 128KB per-block shared-memory histograms past the 48KB cap
- Reached **1.65 TB/s** (49% of H100 peak) with 128-bit `uint4` loads, `__ldg` reads, and Nsight-guided shift/mask tuning

RISC-V DOOM Port | *C, ASM, RISC-V, MangoPi* — Jul. 2025 – Aug. 2025

- Ported DOOM (1993) to bare-metal RISC-V via doomgeneric, embedding the WAD as a C array to replace all file I/O
- Wrote the full platform layer: SPI LCD framebuffer, I2C Joy Bonnet input, I2S soundtrack over DMA, and libc routines
- Added hot-swappable output that polls the HDMI PHY register and remaps the framebuffer (RGB565/RGBA8888) live

LEGO Image Printer | *C, RISC-V, MangoPi, Python* — Jan. 2024 – Mar. 2024

- Built the bare-metal image pipeline in C (downscaling & color quantization) mapping pixels to a limited LEGO palette
- Wrote a Python converter serializing arbitrary images into C bitmap structs, plus the on-device UI for image, scale, and colormap selection with live print-progress feedback (2-person team; partner built the CNC/vacuum hardware)

Technical Skills

Languages: C, C++, CUDA, ISPC, Assembly (RISC-V, ARM, MIPS), Python, Java
Parallel & Performance: Nsight, perf, SIMD/vectorization, shared-memory optimization, cache & roofline analysis
Systems & Embedded: gdb, UART debugging, Make, Git, Linux, KiCad, bare-metal drivers, interrupts, DMA, FAT32
Platforms & Protocols: I2C, I2S, SPI, UART, GPIO; Raspberry Pi, MangoPi (RISC-V), NVIDIA H100; NumPy, scikit-learn